LAPORAN TUGAS BESAR
PROYEK STRUKTUR DATA DAN ALGORITMA

Disusun Oleh :

Khairunnisa Lizal Nurjannah	(G1A025060)
Aurel Clara Belinda Putri	(G1A025070)
Muhammad Zaky Ihsan	(G1A025074)
Azalia Callysta	(G1A025094)
Bunga Gadiza Dwi Bahrizal	(G1A025122)

Nama Asisten Dosen :

1.	I Nyoman Dimas Kresna Adryan	(G1A023077)
2.	Ahsan Nurfauzi	(G1A024002)
3.	Muhammad Iqbal Zafarullah	(G1A024007)
4.	Muhammad Yusuf Arrayyan	(G1A024009)
5.	Shintia Bella	(G1A024014)
6.	Muhammad Kadafi Naufala	(G1A024034)
7.	Alfarian Adisaputra	(G1A024039)
8.	Ali Akbar Bermano	(G1A024041)
9.	Lovien Najla Dhafiyah	(G1A024055)
10.	Gusti Revalion Putrada	(G1A024081)

Dosen Pengampu :

1. Ir. Arie Vatresia, S.T., M.T.I., Ph.D., IPP
2. Ulfasari Rafflesia, S.Si, M.Si

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS BENGKULU

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan karya tulis ini yang berjudul *"Implementasi Struktur Data dan Algoritma dalam Pembuatan Sistem Informasi Inventaris Rumah Sakit Menggunakan Aplikasi Qt"* dengan baik.

Laporan ini disusun dengan tujuan untuk memberikan pemahaman mengenai pentingnya penerapan konsep struktur data dan algoritma dalam pengembangan sistem informasi, khususnya pada pengelolaan inventaris di lingkungan rumah sakit. Sistem inventaris yang baik sangat dibutuhkan untuk meningkatkan efisiensi, akurasi, serta kemudahan dalam pengelolaan data barang, alat medis, dan kebutuhan operasional lainnya.

Dalam pengembangan sistem ini, penulis memanfaatkan aplikasi Qt sebagai framework untuk membangun antarmuka pengguna yang interaktif dan responsif. Selain itu, penerapan struktur data yang tepat serta algoritma yang efisien menjadi faktor utama dalam memastikan sistem dapat berjalan secara optimal, cepat, dan terorganisir.

Penulis menyadari bahwa dalam penyusunan karya tulis ini masih terdapat banyak kekurangan, baik dari segi materi maupun penyajiannya. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi penyempurnaan karya tulis ini di masa yang akan datang.

Akhir kata, penulis berharap semoga karya tulis ini dapat memberikan manfaat bagi pembaca, khususnya bagi mahasiswa dan pengembang yang tertarik dalam bidang sistem informasi, struktur data, dan pemrograman menggunakan Qt.

Bengkulu, 13 April 2026

Kelompok 1

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Dan Manfaat 2
BAB II 3
LANDASAN TEORI 3
BAB III 5
PEMBAHASAN 5
BAB IV 25
KESIMPULAN DAN SARAN 25
A. Kesimpulan 25
B. Saran 25
DAFTAR PUSTAKA 26

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat telah mendorong berbagai instansi, termasuk rumah sakit, untuk meningkatkan kualitas pelayanan melalui pemanfaatan sistem informasi. Salah satu aspek penting dalam operasional rumah sakit adalah pengelolaan inventaris, yang meliputi pencatatan, penyimpanan, serta pemantauan barang dan alat medis. Pengelolaan inventaris yang masih dilakukan secara manual berpotensi menimbulkan berbagai permasalahan, seperti kesalahan pencatatan, kehilangan data, serta keterlambatan dalam pencarian informasi.

Untuk mengatasi permasalahan tersebut, diperlukan suatu sistem informasi inventaris yang terkomputerisasi. Dalam pengembangan sistem ini, konsep struktur data dan algoritma memiliki peran yang sangat penting, terutama dalam pengolahan dan pencarian data. Salah satu algoritma pencarian yang sederhana dan mudah diimplementasikan adalah *linear search*, yang bekerja dengan cara menelusuri data secara berurutan hingga data yang dicari ditemukan.

Selain itu, penggunaan aplikasi Qt sebagai framework pengembangan perangkat lunak memberikan kemudahan dalam membangun antarmuka pengguna (user interface) yang interaktif dan user-friendly. Dengan mengombinasikan struktur data yang tepat, algoritma linear search, dan penggunaan Qt, diharapkan sistem informasi inventaris rumah sakit dapat berjalan secara efektif dan efisien.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang dan membangun sistem informasi inventaris rumah sakit menggunakan aplikasi Qt?
2. Bagaimana penerapan struktur data dalam pengelolaan data inventaris rumah sakit?

3. Bagaimana implementasi algoritma linear search dalam proses pencarian data inventaris?
4. Bagaimana kinerja sistem dalam membantu proses pengelolaan dan pencarian data inventaris?

1.3 Tujuan Dan Manfaat

Tujuan dari penyusunan karya tulis ini adalah:

1. Merancang dan membangun sistem informasi inventaris rumah sakit berbasis aplikasi Qt.
2. Menerapkan konsep struktur data dalam pengelolaan data inventaris secara terstruktur dan efisien.
3. Mengimplementasikan algoritma linear search untuk mempermudah proses pencarian data.
4. Menghasilkan sistem yang mampu meningkatkan efektivitas dan efisiensi pengelolaan inventaris rumah sakit.

Adapun manfaat dari penelitian ini adalah sebagai berikut:

1. Bagi Penulis

   Menambah wawasan dan pemahaman dalam penerapan struktur data dan algoritma, khususnya linear search, dalam pengembangan sistem informasi menggunakan Qt.
2. Bagi Rumah Sakit

   Membantu meningkatkan efisiensi dan akurasi dalam pengelolaan data inventaris serta mempermudah proses pencarian data barang dan alat medis.
3. Bagi Akademisi

   Menjadi referensi atau bahan pembelajaran dalam pengembangan sistem informasi berbasis struktur data dan algoritma.
4. Bagi Pengembang Sistem

   Memberikan gambaran tentang implementasi algoritma sederhana seperti linear search dalam sistem nyata serta penggunaan Qt dalam pengembangan aplikasi desktop, sehingga dapat menjadi dasar dalam pengembangan sistem yang lebih kompleks dan optimal di masa mendatang. Selain itu, hasil penelitian ini juga dapat dijadikan acuan dalam memilih metode pencarian yang sesuai dengan kebutuhan sistem.

BAB II
LANDASAN TEORI

Struktur data merupakan salah satu konsep dasar dalam ilmu komputer yang berfungsi untuk mengorganisasi dan menyimpan data sehingga dapat diakses serta dikelola secara efisien. Pemilihan struktur data yang tepat sangat berpengaruh terhadap performa suatu sistem, terutama dalam hal kecepatan pemrosesan dan penggunaan memori. Dalam pengembangan sistem informasi, struktur data menjadi fondasi utama yang menentukan bagaimana data akan disimpan, diolah, dan ditampilkan kepada pengguna. Struktur data yang umum digunakan dalam sistem inventaris antara lain array dan list, karena mampu menyimpan data secara terurut dan memudahkan proses pencarian serta pengolahan data.

Selain struktur data, algoritma juga memiliki peran yang sangat penting dalam pengembangan sistem informasi. Algoritma merupakan serangkaian langkah logis dan sistematis yang digunakan untuk menyelesaikan suatu permasalahan. Algoritma yang baik harus memiliki tingkat efisiensi yang tinggi serta mudah dipahami dan diimplementasikan. Dalam konteks sistem informasi inventaris rumah sakit, algoritma digunakan untuk mengelola data barang, melakukan proses pencarian, serta menghasilkan informasi yang dibutuhkan secara cepat dan akurat. Keterkaitan antara struktur data dan algoritma sangat erat, karena algoritma yang efisien memerlukan struktur data yang sesuai agar dapat bekerja secara optimal.

Salah satu algoritma pencarian yang sering digunakan dalam sistem sederhana adalah algoritma linear search atau sequential search. Algoritma ini bekerja dengan cara menelusuri setiap elemen data secara berurutan dari awal hingga akhir sampai data yang dicari ditemukan. Kelebihan dari algoritma linear search adalah kemudahannya dalam implementasi serta tidak memerlukan data dalam keadaan terurut. Hal ini menjadikan algoritma ini sangat cocok digunakan dalam sistem dengan jumlah data yang tidak terlalu besar atau dalam kondisi data yang belum terstruktur dengan baik. Meskipun demikian, kelemahan dari algoritma ini terletak pada waktu pencarian yang relatif lebih lama jika jumlah data semakin besar, karena harus memeriksa setiap elemen satu per satu.

Dalam penerapannya pada sistem informasi inventaris rumah sakit, algoritma linear search dapat digunakan untuk mencari data barang berdasarkan

kode, nama, atau kategori tertentu. Proses pencarian yang sederhana ini cukup efektif untuk sistem berskala kecil hingga menengah. Selain itu, penggunaan algoritma ini juga dapat membantu dalam proses pembelajaran dasar mengenai penerapan algoritma dalam sistem nyata. Dengan demikian, algoritma linear search tetap relevan digunakan sebagai solusi awal dalam pengembangan sistem informasi inventaris.

Sistem informasi inventaris rumah sakit merupakan suatu sistem yang dirancang untuk mengelola data barang dan peralatan medis secara terkomputerisasi. Sistem ini mencakup berbagai proses seperti pencatatan data barang, pengelolaan stok, pencarian data, serta pembuatan laporan. Penggunaan sistem informasi dalam pengelolaan inventaris bertujuan untuk meningkatkan efisiensi kerja, mengurangi kesalahan pencatatan, serta menyediakan informasi yang akurat dan tepat waktu. Tanpa adanya sistem yang terkomputerisasi, pengelolaan inventaris cenderung dilakukan secara manual, yang dapat menyebabkan berbagai permasalahan seperti kehilangan data, ketidaksesuaian stok, dan keterlambatan dalam penyajian informasi.

Dalam pengembangan sistem informasi inventaris, diperlukan suatu platform atau framework yang dapat mendukung pembuatan aplikasi secara efektif. Salah satu framework yang dapat digunakan adalah Qt. Qt merupakan framework berbasis C++ yang digunakan untuk mengembangkan aplikasi dengan antarmuka grafis (GUI). Qt memiliki keunggulan dalam hal kemudahan penggunaan, fleksibilitas, serta kemampuan untuk berjalan di berbagai sistem operasi. Dengan menggunakan Qt, pengembang dapat membuat aplikasi yang interaktif dan user-friendly, sehingga memudahkan pengguna dalam mengoperasikan sistem.

Penggunaan Qt dalam sistem informasi inventaris rumah sakit memungkinkan integrasi antara tampilan antarmuka dengan logika program yang mengimplementasikan struktur data dan algoritma. Hal ini memberikan kemudahan dalam proses input data, pencarian data menggunakan algoritma linear search, serta pengelolaan data secara keseluruhan. Dengan demikian, sistem yang dihasilkan tidak hanya memiliki tampilan yang menarik, tetapi juga memiliki performa yang baik dalam pengolahan data. Secara keseluruhan, struktur data, algoritma, dan sistem informasi merupakan komponen yang saling bekaitan.

BAB III

PEMBAHASAN

1. Buatlah sistem informasi sesuai tema dan menerapkan algoritma yang telah ditentukan. GUI dari sistem informasi dibuat menggunakan aplikasi QT.

   SI rumah sakit (linear search)

   Printscreen Source Code

```
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    // Gunakan style Fusion agar tampilan konsisten di semua platform
    a.setStyle("Fusion");

    // Set font default aplikasi
    QFont font("Segoe UI", 10);
    a.setFont(font);

    // ---- Global Stylesheet ----
    a.setStyleSheet(R"(
        QMainWindow {
            background-color: #0f172a;
        }
        QWidget {
            background-color: #0f172a;
            color: #e2e8f0;
            font-family: 'Segoe UI', Arial, sans-serif;
        }
        QScrollBar:vertical {
            background: #1e293b;
            width: 8px;
            border-radius: 4px;
        }
        QScrollBar::handle:vertical {
            background: #38bdf8;
            border-radius: 4px;
            min-height: 20px;
        }
        QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {
            height: 0px;
        }
        QScrollBar:horizontal {
            background: #1e293b;
            height: 8px;
            border-radius: 4px;
        }
        QScrollBar::handle:horizontal {
            background: #38bdf8;
            border-radius: 4px;
        }
        QScrollBar::add-line:horizontal, QScrollBar::sub-line:horizontal {
            width: 0px;
        }
        QToolTip {
            background-color: #1e293b;
            color: #e2e8f0;
            border: 1px solid #38bdf8;
            padding: 4px 8px;
            border-radius: 4px;
        }
    )");

    // ---- Tampilkan halaman Login terlebih dahulu ----
    // LoginWindow ditampilkan sebagai layar pertama.
    // Setelah login berhasil, signal loginBerhasil dikirim dan
    // MainWindow akan dibuat sesuai role (Admin / User).

    LoginWindow *loginWin = new LoginWindow();
    loginWin->setWindowTitle("🏥  SI Rumah Sakit — Login");
    loginWin->show();

    // Pointer ke MainWindow (dibuat saat login sukses)
    MainWindow *mainWin = nullptr;

    // Hubungkan signal loginBerhasil ke lambda yang membuat MainWindow
    QObject::connect(loginWin, &LoginWindow::loginBerhasil,
                     [&](RoleLogin role, const QString &namaUser) {
                         // Buat MainWindow dengan role dan nama pengguna
                         mainWin = new MainWindow(role, namaUser);
                         mainWin->show();

                         // Tutup jendela login
                         loginWin->close();
                         loginWin->deleteLater();
                     });

    return a.exec();
}
```

Kelompok 1

Gambar 1.1 Source Code Main.cpp

Source Code

```
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    // Gunakan style Fusion agar tampilan konsisten di semua platform
```

```
a.setStyle("Fusion");

// Set font default aplikasi
QFont font("Segoe UI", 10);
a.setFont(font);

// ---- Global Stylesheet ----
a.setStyleSheet(R"(
    QMainWindow {
        background-color: #0f172a;
    }
    QWidget {
        background-color: #0f172a;
        color: #e2e8f0;
        font-family: 'Segoe UI', Arial, sans-serif;
    }
    QScrollBar:vertical {
        background: #1e293b;
        width: 8px;
        border-radius: 4px;
    }
    QScrollBar::handle:vertical {
        background: #38bdf8;
        border-radius: 4px;
        min-height: 20px;
    }
    QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {
        height: 0px;
    }
    QScrollBar:horizontal {
        background: #1e293b;
        height: 8px;
```

```
        border-radius: 4px;
    }
    QScrollBar::handle:horizontal {
        background: #38bdf8;
        border-radius: 4px;
    }
    QScrollBar::add-line:horizontal, QScrollBar::sub-line:horizontal {
        width: 0px;
    }
    QToolTip {
        background-color: #1e293b;
        color: #e2e8f0;
        border: 1px solid #38bdf8;
        padding: 4px 8px;
        border-radius: 4px;
    }
)");

// ---- Tampilkan halaman Login terlebih dahulu ----
// LoginWindow ditampilkan sebagai layar pertama.
// Setelah login berhasil, signal loginBerhasil dikirim dan
// MainWindow akan dibuat sesuai role (Admin / User).

LoginWindow *loginWin = new LoginWindow();
loginWin->setWindowTitle("  SI Rumah Sakit — Login");
loginWin->show();

// Pointer ke MainWindow (dibuat saat login sukses)
MainWindow *mainWin = nullptr;

// Hubungkan signal loginBerhasil ke lambda yang membuat MainWindow
QObject::connect(loginWin, &LoginWindow::loginBerhasil,
```

```
            [&](RoleLogin role, const QString &namaUser) {
                // Buat MainWindow dengan role dan nama pengguna
                mainWin = new MainWindow(role, namaUser);
                mainWin->show();

                // Tutup jendela login
                loginWin->close();
                loginWin->deleteLater();
            });

    return a.exec();
}
```

Penjelasan Source Code:

Kode tersebut merupakan fungsi utama (main) yang menjadi titik awal jalannya aplikasi berbasis Qt. Saat program dijalankan, bagian pertama yang dilakukan adalah membuat objek QApplication yang berfungsi sebagai inti pengelola seluruh aplikasi, termasuk event, tampilan, dan interaksi pengguna. Selanjutnya, aplikasi diatur agar menggunakan style "Fusion". Tujuan dari pengaturan ini adalah supaya tampilan aplikasi terlihat konsisten di berbagai sistem operasi, sehingga tidak bergantung pada tampilan bawaan OS tertentu. Setelah itu, program mengatur font default aplikasi menggunakan jenis huruf *Segoe UI* dengan ukuran 10. Hal ini dilakukan agar seluruh teks dalam aplikasi memiliki tampilan yang rapi, modern, dan seragam. Kemudian, diterapkan global stylesheet (QSS) yang berfungsi untuk mengatur tampilan visual aplikasi secara keseluruhan. Pada bagian ini diatur warna latar belakang utama yang cenderung gelap (dark mode), warna teks, serta tampilan komponen seperti scrollbar dan tooltip. Dengan adanya stylesheet ini, aplikasi menjadi lebih menarik dan memiliki desain yang konsisten. Setelah pengaturan tampilan selesai, program menampilkan jendela login dengan membuat objek LoginWindow. Jendela ini diberi judul "SI Rumah Sakit — Login" dan langsung ditampilkan sebagai halaman pertama yang dilihat oleh pengguna. Selanjutnya, dibuat sebuah pointer mainWin yang nantinya akan digunakan

untuk menyimpan objek MainWindow. Namun, objek ini belum dibuat di awal karena hanya akan muncul setelah pengguna berhasil login. Program kemudian menghubungkan

signal loginBerhasil dari LoginWindow dengan sebuah fungsi lambda. Ketika

pengguna berhasil login, signal tersebut akan mengirimkan data berupa role pengguna (Admin atau User) dan nama pengguna. Data ini kemudian digunakan untuk membuat MainWindow yang sesuai dengan hak akses pengguna.Setelah MainWindow ditampilkan, jendela login akan ditutup dan dihapus dari memori menggunakan deleteLater() agar tidak terjadi kebocoran memori. Terakhir, fungsi a.exec() dijalankan untuk memulai event loop, yaitu proses yang membuat aplikasi tetap berjalan dan siap menerima berbagai interaksi dari pengguna seperti klik tombol, input data, dan lain-lain.

Printscreen Source Code

```
enum class RoleLogin {
    Admin,   // Bisa CRUD semua data
    User     // Hanya bisa melihat (Read-only)
};

struct Pasien {
    int      id;
    QString nama;
    QString jenisKelamin;    // "Laki-laki" / "Perempuan"
    int      umur;
    QDate    tanggalLahir;   // Tanggal lahir pasien (digunakan sebagai password login
user
    QString noTelepon;
    QString alamat;
    QString golonganDarah;
    QString diagnosa;
    QString dokter;
    QString ruangan;
    QDate    tanggalMasuk;
    QString status;          // "Rawat Inap" / "Rawat Jalan" / "Selesai"
};

struct Dokter {
    int      id;
    QString nama;
    QString spesialisasi;
    QString noTelepon;
    QString jadwal;
    bool     tersedia;
};

struct Obat {
    int      id;
    QString nama;
    QString jenis;
    int      stok;
    QString satuan;
    double   harga;
};

/**
 * @brief Struct AkunUser — data akun pengguna (role User, bukan Admin)
 *        Kunci masuk: nama (sebagai username) + tanggal lahir
 *        Data bersumber dari dataPasien yang disimpan di JSON.
 */
struct AkunUser {
    QString username;        // Nama pasien (lowercase, tanpa spasi) atau nama lengkap
    QDate    tanggalLahir;   // Dijadikan sebagai "password" untuk user
    QString namaLengkap;
};

class LoginWindow : public QWidget
{
    Q_OBJECT

public:
    explicit LoginWindow(QWidget *parent = nullptr);

signals:
    // Dikirim ketika login berhasil, membawa role dan nama pengguna
    void loginBerhasil(RoleLogin role, const QString &namaUser);

private slots:
    void prosesLoginAdmin();     // Validasi kredensial admin
    void prosesLoginUser();      // Validasi username + tanggal lahir user
    void gantiBtnLogin();        // Ganti tampilan form sesuai pilihan radio

private:
    void setupUI();
    void setupFormAdmin();
    void setupFormUser();

    // Membaca data pasien dari file JSON yang disimpan MainWindow
    // dan mengubahnya menjadi daftar AkunUser untuk validasi login
    QVector<AkunUser> bacaAkunUserDariJSON() const;
    static QString    pathDataFile();    // Path file JSON (sama dengan MainWindow)

    // ----- Widget utama -----
    QStackedWidget *stackForm;       // Berganti antara form admin / user
    QWidget        *formAdmin;
    QWidget        *formUser;

    // Form Admin
    QLineEdit      *inputAdminUsername;
    QLineEdit      *inputAdminPassword;

    // Form User
    QLineEdit      *inputUserUsername;
    QDateEdit      *inputUserTanggalLahir;

    // Pilihan role (radio button)
    QRadioButton   *radioAdmin;
    QRadioButton   *radioUser;

    // Kredensial admin universal (hardcoded)
    const QString ADMIN_USERNAME = "admin";
    const QString ADMIN_PASSWORD = "admin123";
};
```

Kelompok 1

```cpp
// ==========================================================================
// MAIN WINDOW — Tampil setelah login berhasil
// ==========================================================================

/**
 * @brief MainWindow — Jendela utama aplikasi SI Rumah Sakit.
 *        Menampilkan halaman berbeda tergantung role:
 *        - Admin : bisa akses CRUD semua data
 *        - User  : hanya bisa melihat data (tombol tambah/edit/hapus disembunyikan)
 */
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    /**
     * @param role       Role yang login (Admin atau User)
     * @param namaUser   Nama pengguna yang ditampilkan di sidebar
     * @param parent     Parent widget
     */
    explicit MainWindow(RoleLogin role, const QString &namaUser, QWidget *parent =
nullptr);
    ~MainWindow();

private slots:
    // Navigasi antar halaman
    void showDashboard();
    void showPasienPage();
    void showDokterPage();
    void showObatPage();
    void showCariPage();
    void prosesLogout();          // Kembali ke halaman login

    // Pasien CRUD (hanya bisa diakses Admin)
    void tambahPasien();
    void editPasien();
    void hapusPasien();
    void refreshTabelPasien();

    // Dokter CRUD (hanya bisa diakses Admin)
    void tambahDokter();
    void editDokter();
    void hapusDokter();
    void refreshTabelDokter();

    // Obat CRUD (hanya bisa diakses Admin)
    void tambahObat();
    void editObat();
    void hapusObat();
    void refreshTabelObat();

    // Linear Search
    void cariPasien();
    void cariDokter();
    void cariObat();
    void onSearchTypeChanged(int index);

    // Dashboard & Clock
    void updateDashboard();
    void updateClock();

private:
    // ---- Setup UI ----
    void setupUI();
    void setupSidebar();
    void setupDashboardPage();
    void setupPasienPage();
    void setupDokterPage();
    void setupObatPage();
    void setupCariPage();

    // ---- Helper ----
    QWidget*        createStatCard(const QString &title, const QString &value,
                                   const QString &icon, const QString &color);
    QPushButton* createNavButton(const QString &text, const QString &icon);
    void         applyTableStyle(QTableWidget *table);
    void         setActiveNavButton(QPushButton *btn);
    void         inisialisasiDataContoh();

    // ---- Linear Search Algorithm ----
    QVector<int> linearSearchPasien(const QString &keyword, const QString &field);
    QVector<int> linearSearchDokter(const QString &keyword, const QString &field);
    QVector<int> linearSearchObat  (const QString &keyword, const QString &field);

    // ---- Persistensi Data (JSON) ----
    void    simpanData();
    void    muatData();
    QString pathDataFile() const;

    // ---- Widgets ----
    QWidget     *centralWidget;
    QHBoxLayout *mainLayout;

    // Sidebar
    QWidget     *sidebar;
    QVBoxLayout *sidebarLayout;
    QPushButton *btnDashboard;
    QPushButton *btnPasien;
    QPushButton *btnDokter;
    QPushButton *btnObat;
    QPushButton *btnCari;
    QPushButton *btnLogout;
    QPushButton *activeNavBtn;
    QLabel      *lblInfoUser;      // Menampilkan nama & role yang login

    // Stacked pages
    QStackedWidget *stackedWidget;
    QWidget        *pageDashboard;
    QWidget        *pagePasien;
    QWidget        *pageDokter;
    QWidget        *pageObat;
    QWidget        *pageCari;

    // Dashboard widgets
    QLabel         *lblJamTanggal;
    QLabel         *lblTotalPasien;
    QLabel         *lblPasienRawatInap;
    QLabel         *lblPasienRawatJalan;
    QLabel         *lblTotalDokter;
    QLabel         *lblTotalObat;

    // Pasien widgets
    QTableWidget   *tabelPasien;
    QLineEdit      *inputCariPasienPage;

    // Dokter widgets
    QTableWidget   *tabelDokter;
    QLineEdit      *inputCariDokterPage;

    // Obat widgets
    QTableWidget   *tabelObat;
    QLineEdit      *inputCariObatPage;

    // Cari (Search) widgets
    QComboBox      *comboSearchType;
    QComboBox      *comboSearchField;
    QLineEdit      *inputSearchKeyword;
    QTableWidget   *tabelHasilCari;
    QLabel         *lblHasilInfo;
    QLabel         *lblStepInfo;

    // Data — disimpan sebagai array (QVector)
    QVector<Pasien> dataPasien;
    QVector<Dokter> dataDokter;
    QVector<Obat>   dataObat;
    int             nextPasienId;
    int             nextDokterId;
    int             nextObatId;

    // Timer untuk jam di dashboard
    QTimer          *clockTimer;

    // Role pengguna yang sedang login
    RoleLogin       roleAktif;
    QString         namaUserAktif;
};
```

Kelompok 1

Gambar 1.2 Source Code Mainwindow.h

Source Code

```
enum class RoleLogin {
    Admin,  // Bisa CRUD semua data
    User    // Hanya bisa melihat (Read-only)
};
struct Pasien {
    int     id;
    QString nama;
    QString jenisKelamin;   // "Laki-laki" / "Perempuan"
    int     umur;
    QDate   tanggalLahir;   // Tanggal lahir pasien (digunakan sebagai
password login user)
    QString noTelepon;
    QString alamat;
    QString golonganDarah;
    QString diagnosa;
    QString dokter;
    QString ruangan;
    QDate   tanggalMasuk;
    QString status;         // "Rawat Inap" / "Rawat Jalan" / "Selesai"
};

/**
 * @brief Struct Dokter — menyimpan data satu dokter
 */
struct Dokter {
    int     id;
    QString nama;
    QString spesialisasi;
    QString noTelepon;
    QString jadwal;
    bool    tersedia;
```

```
};

/**
 * @brief Struct Obat — menyimpan data satu jenis obat
 */
struct Obat {
    int     id;
    QString nama;
    QString jenis;
    int     stok;
    QString satuan;
    double  harga;
};

 * @brief Struct AkunUser — data akun pengguna (role User, bukan Admin)
 *        Kunci masuk: nama (sebagai username) + tanggal lahir
 *        Data bersumber dari dataPasien yang disimpan di JSON.
struct AkunUser {
    QString username;       // Nama pasien (lowercase, tanpa spasi) atau nama
lengkap
    QDate   tanggalLahir;   // Dijadikan sebagai "password" untuk user
    QString namaLengkap;
};
/**
 * @brief LoginWindow — Jendela login dengan dua opsi:
 *        1. Login sebagai User   → username + tanggal lahir
 *        2. Login sebagai Admin  → username universal + password universal
 *
 *  Credentials Admin (universal):
 *      Username : admin
 *      Password : admin123
 *
```

```
class LoginWindow : public QWidget
{
    Q_OBJECT

public:
    explicit LoginWindow(QWidget *parent = nullptr);

signals:
    // Dikirim ketika login berhasil, membawa role dan nama pengguna
    void loginBerhasil(RoleLogin role, const QString &namaUser);

private slots:
    void prosesLoginAdmin();    // Validasi kredensial admin
    void prosesLoginUser();     // Validasi username + tanggal lahir user
    void gantiBtnLogin();       // Ganti tampilan form sesuai pilihan radio

private:
    void setupUI();
    void setupFormAdmin();
    void setupFormUser();

    // Membaca data pasien dari file JSON yang disimpan MainWindow
    // dan mengubahnya menjadi daftar AkunUser untuk validasi login
    QVector<AkunUser> bacaAkunUserDariJSON() const;
    static QString    pathDataFile();   // Path file JSON (sama dengan
MainWindow)

    // ---- Widget utama ----
    QStackedWidget  *stackForm;     // Berganti antara form admin / user
    QWidget         *formAdmin;
    QWidget         *formUser;
    // Form Admin
```

```
    QLineEdit       *inputAdminUsername;
    QLineEdit       *inputAdminPassword;

    // Form User
    QLineEdit       *inputUserUsername;
    QDateEdit       *inputUserTanggalLahir;

    // Pilihan role (radio button)
    QRadioButton    *radioAdmin;
    QRadioButton    *radioUser;

    // Kredensial admin universal (hardcoded)
    const QString ADMIN_USERNAME = "admin";
    const QString ADMIN_PASSWORD = "admin123";
};

* @brief MainWindow — Jendela utama aplikasi SI Rumah Sakit.
 *       Menampilkan halaman berbeda tergantung role:
 *       - Admin : bisa akses CRUD semua data
 *       - User  : hanya bisa melihat data (tombol tambah/edit/hapus
disembunyikan)
 */
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    /**
     * @param role      Role yang login (Admin atau User)
     * @param namaUser  Nama pengguna yang ditampilkan di sidebar
     * @param parent    Parent widget
     */
```

```
    explicit MainWindow(RoleLogin role, const QString &namaUser,
QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    // Navigasi antar halaman
    void showDashboard();
    void showPasienPage();
    void showDokterPage();
    void showObatPage();
    void showCariPage();
    void prosesLogout();        // Kembali ke halaman login

    // Pasien CRUD (hanya bisa diakses Admin)
    void tambahPasien();
    void editPasien();
    void hapusPasien();
    void refreshTabelPasien();

    // Dokter CRUD (hanya bisa diakses Admin)
    void tambahDokter();
    void editDokter();
    void hapusDokter();
    void refreshTabelDokter();

    // Obat CRUD (hanya bisa diakses Admin)
    void tambahObat();
    void editObat();
    void hapusObat();
    void refreshTabelObat();

    // Linear Search
```

```
    void cariPasien();
    void cariDokter();
    void cariObat();
    void onSearchTypeChanged(int index);

    // Dashboard & Clock
    void updateDashboard();
    void updateClock();

private:
    // ---- Setup UI ----
    void setupUI();
    void setupSidebar();
    void setupDashboardPage();
    void setupPasienPage();
    void setupDokterPage();
    void setupObatPage();
    void setupCariPage();

    // ---- Helper ----
    QWidget*     createStatCard(const QString &title, const QString &value,
                        const QString &icon, const QString &color);
    QPushButton* createNavButton(const QString &text, const QString
&icon);
    void         applyTableStyle(QTableWidget *table);
    void         setActiveNavButton(QPushButton *btn);
    void         inisialisasiDataContoh();

    // ---- Linear Search Algorithm ----
    QVector<int> linearSearchPasien(const QString &keyword, const QString
&field);
```

```
    QVector<int> linearSearchDokter(const QString &keyword, const QString
&field);
    QVector<int> linearSearchObat  (const QString &keyword, const QString
&field);

    // ---- Persistensi Data (JSON) ----
    void    simpanData();
    void    muatData();
    QString pathDataFile() const;

    // ---- Widgets ----
    QWidget       *centralWidget;
    QHBoxLayout   *mainLayout;

    // Sidebar
    QWidget       *sidebar;
    QVBoxLayout   *sidebarLayout;
    QPushButton   *btnDashboard;
    QPushButton   *btnPasien;
    QPushButton   *btnDokter;
    QPushButton   *btnObat;
    QPushButton   *btnCari;
    QPushButton   *btnLogout;
    QPushButton   *activeNavBtn;
    QLabel        *lblInfoUser;     // Menampilkan nama & role yang login

    // Stacked pages
    QStackedWidget *stackedWidget;
    QWidget       *pageDashboard;
    QWidget       *pagePasien;
    QWidget       *pageDokter;
    QWidget       *pageObat;
```

```
QWidget       *pageCari;

// Dashboard widgets
QLabel        *lblJamTanggal;
QLabel        *lblTotalPasien;
QLabel        *lblPasienRawatInap;
QLabel        *lblPasienRawatJalan;
QLabel        *lblTotalDokter;
QLabel        *lblTotalObat;

// Pasien widgets
QTableWidget   *tabelPasien;
QLineEdit      *inputCariPasienPage;

// Dokter widgets
QTableWidget   *tabelDokter;
QLineEdit      *inputCariDokterPage;

// Obat widgets
QTableWidget   *tabelObat;
QLineEdit      *inputCariObatPage;

// Cari (Search) widgets
QComboBox      *comboSearchType;
QComboBox      *comboSearchField;
QLineEdit      *inputSearchKeyword;
QTableWidget   *tabelHasilCari;
QLabel        *lblHasilInfo;
QLabel        *lblStepInfo;

// Data — disimpan sebagai array (QVector)
QVector<Pasien> dataPasien;
```

```
    QVector<Dokter> dataDokter;
    QVector<Obat>   dataObat;
    int             nextPasienId;
    int             nextDokterId;
    int             nextObatId;
    // Timer untuk jam di dashboard
    QTimer          *clockTimer;
    // Role pengguna yang sedang login
    RoleLogin       roleAktif;
    QString         namaUserAktif;
};
```

Penjelasan Source Code:

Pada bagian awal terdapat enum RoleLogin yang berfungsi untuk menentukan hak akses pengguna. Admin memiliki akses penuh untuk mengelola data seperti menambah, mengedit, dan menghapus, sedangkan user hanya dapat melihat data tanpa bisa mengubahnya. Konsep ini disebut sebagai pembatasan hak akses berdasarkan peran pengguna. Selanjutnya terdapat beberapa struktur data atau struct, yaitu Pasien, Dokter, Obat, dan AkunUser. Struct Pasien digunakan untuk menyimpan seluruh informasi pasien seperti nama, umur, alamat, diagnosa, dan status perawatan. Struct Dokter menyimpan data dokter seperti nama, spesialisasi, jadwal, dan ketersediaan. Struct Obat berisi informasi obat seperti nama, stok, dan harga. Sedangkan struct AkunUser digunakan untuk keperluan login user, di mana username diambil dari nama pasien dan password menggunakan tanggal lahir pasien. Class LoginWindow berfungsi sebagai tampilan awal aplikasi untuk proses login. Di dalamnya terdapat dua metode login, yaitu login sebagai admin dan login sebagai user. Login admin menggunakan username dan password tetap yang sudah ditentukan dalam program, sedangkan login user menggunakan data pasien yang diambil dari file JSON. Jika login berhasil, maka program akan mengirimkan sinyal yang berisi informasi role dan nama pengguna untuk membuka tampilan utama. Class MainWindow merupakan inti dari aplikasi yang menampilkan berbagai halaman seperti dashboard, data pasien, data

dokter, data obat, dan fitur pencarian. Navigasi antar halaman diatur menggunakan fungsi-fungsi khusus. Pada bagian ini juga terdapat fungsi CRUD yang memungkinkan admin untuk menambah, mengedit, dan menghapus data pasien, dokter, dan obat. Data-data tersebut disimpan dalam struktur QVector yang berfungsi sebagai array dinamis. Selain itu, program juga memiliki fitur pencarian data menggunakan algoritma linear search, yaitu mencari data satu per satu dari awal hingga ditemukan hasil yang sesuai. Data aplikasi disimpan ke dalam file JSON agar dapat digunakan kembali saat program dijalankan lagi. Terdapat juga fitur tambahan seperti dashboard yang menampilkan informasi jumlah data serta jam real-time menggunakan timer. Secara keseluruhan, program ini menerapkan konsep pemrograman berorientasi objek, struktur data, algoritma pencarian, serta pengelolaan data berbasis file, sehingga dapat digunakan sebagai sistem sederhana untuk membantu pengelolaan data di rumah sakit.

Printscreen Source Code

```
void MainWindow::tambahDokter()
{
    if (roleAktif != RoleLogin::Admin) return;

    DialogDokter dlg(this);
    if (dlg.exec() == QDialog::Accepted) {
        Dokter d = dlg.getDokter();
        d.id = nextDokterId++;
        dataDokter.append(d);

        refreshTabelDokter();
        simpanData();
    }
}
void MainWindow::editDokter()
{
    if (roleAktif != RoleLogin::Admin) return;

    int row = tabelDokter->currentRow();
    if (row < 0) return;

    int id = tabelDokter->item(row,0)->text().toInt();

    for (int i = 0; i < dataDokter.size(); ++i) {
        if (dataDokter[i].id == id) {

            DialogDokter dlg(this, &dataDokter[i]);
            if (dlg.exec() == QDialog::Accepted) {
                dataDokter[i] = dlg.getDokter();
                refreshTabelDokter();
                simpanData();
            }
            break;
        }
    }
}
void MainWindow::hapusDokter()
{
    if (roleAktif != RoleLogin::Admin) return;

    int row = tabelDokter->currentRow();
    if (row < 0) return;

    int id = tabelDokter->item(row,0)->text().toInt();

    for (int i = 0; i < dataDokter.size(); ++i)
        if (dataDokter[i].id == id) {
            dataDokter.remove(i);
            break;
        }

    refreshTabelDokter();
    simpanData();
}
QVector<int> MainWindow::linearSearchDokter(const QString &keyword, const QString
&field)
    QVector<int> hasil;

    for (int i = 0; i < dataDokter.size(); ++i) {
        if (dataDokter[i].nama.contains(keyword, Qt::CaseInsensitive
            hasil.append(i);
```

Kelompok 1

```
        }
    }
    return hasil;
}
Dokter DialogDokter::getDokter() const
{
    Dokter d;
    d.nama = inputNama->text().trimmed();
    d.spesialisasi = inputSpesialisasi->text().trimmed();
    d.noTelepon = inputTelepon->text().trimmed();
    d.jadwal = inputJadwal->text().trimmed();
    d.tersedia = comboTersedia->currentIndex() == 0;
    return d;
}
```

Kelompok 1

Gambar 1.3 Source Code MainWindow.cpp

Soure Code

```
void MainWindow::tambahDokter()
{
    if (roleAktif != RoleLogin::Admin) return;

    DialogDokter dlg(this);
    if (dlg.exec() == QDialog::Accepted) {
        Dokter d = dlg.getDokter();
        d.id = nextDokterId++;
        dataDokter.append(d);

        refreshTabelDokter();
        simpanData();
    }
}
void MainWindow::editDokter()
{
    if (roleAktif != RoleLogin::Admin) return;

    int row = tabelDokter->currentRow();
    if (row < 0) return;

    int id = tabelDokter->item(row,0)->text().toInt();

    for (int i = 0; i < dataDokter.size(); ++i) {
```

```
        if (dataDokter[i].id == id) {

            DialogDokter dlg(this, &dataDokter[i]);
            if (dlg.exec() == QDialog::Accepted) {
                dataDokter[i] = dlg.getDokter();
                refreshTabelDokter();
                simpanData();
            }
            break;
        }
    }
}
void MainWindow::hapusDokter()
{
    if (roleAktif != RoleLogin::Admin) return;

    int row = tabelDokter->currentRow();
    if (row < 0) return;

    int id = tabelDokter->item(row,0)->text().toInt();

    for (int i = 0; i < dataDokter.size(); ++i)
        if (dataDokter[i].id == id) {
            dataDokter.remove(i);
            break;
        }

    refreshTabelDokter();
    simpanData();
}
QVector<int> MainWindow::linearSearchDokter(const QString &keyword,
const QString &field)
```

```
{
    QVector<int> hasil;

    for (int i = 0; i < dataDokter.size(); ++i) {
        if (dataDokter[i].nama.contains(keyword, Qt::CaseInsensitive)) {
            hasil.append(i);
        }
    }
    return hasil;
}
Dokter DialogDokter::getDokter() const
{
    Dokter d;
    d.nama = inputNama->text().trimmed();
    d.spesialisasi = inputSpesialisasi->text().trimmed();
    d.noTelepon = inputTelepon->text().trimmed();
    d.jadwal = inputJadwal->text().trimmed();
    d.tersedia = comboTersedia->currentIndex() == 0;
    return d;
}
```

Penjelasan Source Code

Kode tersebut merupakan bagian dari fitur pengelolaan data dokter pada aplikasi Sistem Informasi Rumah Sakit, yang mencakup proses penambahan, pengeditan, penghapusan, pencarian, serta pengambilan data dari form input. Pada fungsi `tambahDokter()`, program digunakan untuk menambahkan data dokter baru. Pertama, sistem akan mengecek apakah pengguna yang sedang login adalah admin. Jika bukan admin, maka proses langsung dihentikan. Jika pengguna adalah admin, maka akan ditampilkan dialog input data dokter. Setelah pengguna mengisi data dan menekan tombol OK, data dokter akan diambil dari dialog, kemudian diberikan ID secara otomatis menggunakan variabel `nextDokterId` yang terus bertambah. Data tersebut kemudian disimpan ke dalam array `dataDokter`. Setelah itu, tabel dokter akan diperbarui

agar menampilkan data terbaru, dan data juga disimpan ke dalam file. Pada fungsi `editDokter()`, program digunakan untuk mengubah data dokter yang sudah ada. Sistem terlebih dahulu memastikan bahwa pengguna adalah admin. Kemudian program mengambil baris data yang sedang dipilih pada tabel. Jika tidak ada baris yang dipilih, maka proses dihentikan. Selanjutnya, program mengambil ID dokter dari tabel, lalu mencari data dokter tersebut di dalam array `dataDokter`. Jika ditemukan, akan ditampilkan dialog yang berisi data dokter tersebut. Setelah data diedit dan disimpan, data lama akan diganti dengan data baru, kemudian tabel diperbarui dan data disimpan kembali. Pada fungsi `hapusDokter()`, program digunakan untuk menghapus data dokter. Sama seperti sebelumnya, hanya admin yang dapat melakukan aksi ini. Program akan mengambil baris yang dipilih pada tabel, lalu mengambil ID dokter. Setelah itu, program mencari data dokter dengan ID tersebut di dalam array dan menghapusnya. Setelah penghapusan, tabel diperbarui dan data disimpan kembali agar perubahan tersimpan permanen. Fungsi `linearSearchDokter()` digunakan untuk melakukan pencarian data dokter menggunakan algoritma linear search. Program akan memeriksa setiap data dokter satu per satu dari awal hingga akhir. Terakhir, fungsi `getDokter()` pada class `DialogDokter` digunakan untuk mengambil data yang telah diinput oleh pengguna melalui form. Data seperti nama, spesialisasi, nomor telepon, dan jadwal diambil dari input field, kemudian dibersihkan dari spasi berlebih menggunakan fungsi `trimmed()`. Status ketersediaan dokter ditentukan berdasarkan pilihan pada combo box. Semua data tersebut kemudian dimasukkan ke dalam struct `Dokter` dan dikembalikan untuk digunakan oleh fungsi lain seperti tambah atau edit data. Selain itu, keseluruhan kode ini menunjukkan penerapan konsep pemrograman berorientasi objek (Object-Oriented Programming), di mana setiap fungsi memiliki tanggung jawab yang jelas dalam mengelola data dokter. Penggunaan class seperti `MainWindow` dan `DialogDokter` membantu memisahkan antara logika program dan tampilan antarmuka, sehingga kode menjadi lebih terstruktur, mudah dipahami, dan mudah dikembangkan di masa depan. Di sisi lain, kode ini juga menerapkan pengelolaan data secara dinamis menggunakan `QVector`.

BAB IV
KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil pembahasan mengenai penerapan struktur data dan algoritma dalam pembuatan sistem informasi inventaris rumah sakit menggunakan aplikasi Qt, dapat disimpulkan bahwa penggunaan teknologi informasi sangat penting dalam meningkatkan efisiensi dan efektivitas pengelolaan data inventaris. Sistem yang sebelumnya dilakukan secara manual memiliki berbagai kelemahan, seperti rentan terhadap kesalahan pencatatan, keterlambatan dalam pencarian data, serta kurangnya akurasi informasi yang dihasilkan.

Dengan diterapkannya struktur data yang tepat, data inventaris dapat disimpan dan dikelola secara lebih terstruktur sehingga memudahkan dalam proses pengolahan dan pencarian data. Selain itu, penggunaan algoritma linear search sebagai metode pencarian data memberikan solusi yang sederhana dan mudah diimplementasikan, khususnya untuk sistem dengan jumlah data yang tidak terlalu besar. Meskipun algoritma ini memiliki keterbatasan dalam hal efisiensi pada data berskala besar, namun dalam konteks sistem inventaris rumah sakit yang sederhana, algoritma ini masih cukup efektif untuk digunakan.

B. Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat diberikan untuk pengembangan sistem selanjutnya. Pertama, sistem informasi inventaris ini dapat dikembangkan lebih lanjut dengan menggunakan algoritma pencarian yang lebih efisien, seperti binary search atau metode pencarian lainnya, terutama jika jumlah data yang dikelola semakin besar. Hal ini bertujuan untuk meningkatkan kecepatan dan performa sistem dalam proses pencarian data.

Kedua, pengembangan sistem dapat ditingkatkan dengan menambahkan fitur-fitur tambahan seperti laporan otomatis, notifikasi stok barang, serta integrasi dengan database yang lebih kompleks agar sistem mampu menangani Data dalam skala yang lebih besar dan dinamis.

DAFTAR PUSTAKA

Rizkyanto, A. A., Laksana, T. G., & Mukhlis. (2024). Penerapan algoritma linear search dalam sistem informasi penjualan sepatu berbasis web. *Jurnal Ilmiah Multidisiplin*, 2(1), 15–29.

Fernandez, S., Mahfuzhi, A. R., & Rabbani, B. (2024). Pemanfaatan algoritma sequential search dalam penerapan aplikasi inventaris. *Journal of Electrical Engineering and Computer (JEECOM)*, 6(1).

Ramadhan, A. (2023). Pengembangan aplikasi rumah sakit menggunakan algoritma sequential search. *Infomatek*, 24(2).

Sitorus, L., Saragih, J. L., & Sihombing, T. A. E. M. (2023). Sistem informasi inventaris barang berbasis website pada rumah sakit. *Jurnal Komputer dan Informatika (JUKI)*, 4(2).

Supriatna, A. D., Rahayu, S., & Rozi, A. F. (2022). Perancangan sistem informasi inventaris barang berbasis web menggunakan metode RAD. *Jurnal Algoritma*, 19(1), 228–238.

Trijayanti, A., Aulia, I., Khairunisa, N., Purba, F. A. H., & Gunawan, I. (2025). Implementasi struktur data dalam sistem komputer. *Jurnal Publikasi Teknik Informatika*, 4(2), 48–53.

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI REPUBLIK INDONESIA
UNIVERSITAS BENGKULU
FAKULTAS TEKNIK
PROGRAM STUDI INFORMATIKA
Jl. Wr. Supratman Kandang Limun, Bengkulu Bengkulu 38371 A Telp: (0736) 344087, 22105 - 227

LEMBAR ASISTENSI
PROYEK STRUKTUR DATA DAN ALGORITMA

Nama : Kelompok 1

Dosen : 1. Ir. Arie Vatresia, S.T., M.T.I., Ph.D., IPP
2. Ulfasari Rafflesia, S.Si, M.Si

Asisten Dosen : 1. I Nyoman Dimas Kresna Adryan (G1A023077)
2. Ahsan Nurfauzi (G1A024002)
3. Muhammad Iqbal Zafarullah (G1A024007)
4. Muhammad Yusuf Arrayyan (G1A024009)
5. Shintia Bella (G1A024014)
6. Muhammad Kadafi Naufala (G1A024034)
7. Alfarian Adisaputra (G1A024039)
8. Ali Akbar Bermano (G1A024041)
9. Lovien Najla Dhafiyah (G1A024055)
10. Gusti Revalion Putrada (G1A024081)

Laporan Praktikum	Catatan dan Tanda Tangan
Laporan Tugas Besar	